

University of Wolverhampton

School of Mathematics and Computer Science

5CS022 Distribute and Cloud Systems Programming

Workshop 3 The Akka Framework

Overview

This workshop continues the exploration of the Akka Framework from the previous tutorial.

1. The Actor class Actors in the sample Akka program currently responds to only one message object – MessageA. Modify the createReceive() method so that it will also respond to any other message and print out the message on the standard output.

Program:

```
package com.example;

import akka.actor.AbstractActor;
import akka.actor.ActorRef;
import akka.actor.Props;

public class ActorA extends AbstractActor {

    public static Props props() {
        return Props.create(ActorA.class, ActorA::new);
    }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
            .match(Integer.class, this::onMessageA)
            .match(Character.class, this::onMessageB)
            .matchAny(this::onAnyOtherMessage)
            .build();
    }

    private void onMessageA(Integer msg) {
        System.out.println("Actor A received Message A: " + msg + " from " + getSender());
        if (msg == 0) {
            getContext().getSystem().terminate();
        } else {
            ActorRef actorBRef = getContext().getSystem().actorOf(Props.create(ActorB.class));
            actorBRef.tell(10, getSelf());
        }
        for (int i = 0; i < 10; i++) {
            System.out.println("Actor A doing work " + i);
        }
    }

    private void onMessageB(Character msg) {
        System.out.println("Actor A received Message B: " + msg + " from " + getSender());
        getSender().tell('e', getSelf());
        for (int i = 0; i < 10; i++) {
            System.out.println("Actor A doing more work " + i);
        }
    }

    private void onAnyOtherMessage(Object msg) {
        System.out.println("Actor A received an unknown message: " + msg + " from " + getSender());
    }
}
```

```
Main (3) [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (Mar 24, 2025, 1:08:36 PM)
[2025-03-24 13:08:36,964] [INFO] [akka.event.slf4j.Slf4jLogger] [default-akka-actor] Actor A received an unknown message: com.example.MessageA@22deb860 f
```

Explanation: This Akka actor, ActorA, is designed to handle three types of messages. If it receives an Integer, it processes the message in the onMessageA method, where it prints the message and, if the value is 0, terminates the actor system. Otherwise, it creates a new actor (ActorB) and sends it a message. If ActorA receives a Character, it processes it in onMessageB, prints the message, responds with 'e', and performs additional work. For any other message type, the onAnyOtherMessage method logs the received message, ensuring the actor handles unexpected inputs gracefully.

2. Every message in Akka is a Java object. However, not all messages need to have a custom Java class created for them. Convert the sample program so that it can respond to messages that contain the primitive data types such as byte, short, int, long, float, double, boolean, and char, without having to create custom Java classes.

Program:

```
@Override
public Receive createReceive() {
    return receiveBuilder()
        .match(Integer.class, this::onMessage)
        .match(Byte.class, this::onMessage)
        .match(Short.class, this::onMessage)
        .match(Long.class, this::onMessage)
        .match(Float.class, this::onMessage)
        .match(Double.class, this::onMessage)
        .match(Boolean.class, this::onMessage)
        .match(Character.class, this::onMessage)
        .matchAny(this::onAnyOtherMessage)
        .build();
}

private void onMessage(Object msg) {
    if (msg instanceof Integer) {
        System.out.println("Actor A received Integer: " + msg);
    } else if (msg instanceof Boolean) {
        System.out.println("Actor A received Boolean: " + msg);
    } else if (msg instanceof Byte) {
        System.out.println("Actor A received Byte: " + msg);
    } else if (msg instanceof Short) {
        System.out.println("Actor A received Short: " + msg);
    } else if (msg instanceof Long) {
        System.out.println("Actor A received Long: " + msg);
    } else if (msg instanceof Float) {
        System.out.println("Actor A received Float: " + msg);
    } else if (msg instanceof Double) {
        System.out.println("Actor A received Double: " + msg);
    } else if (msg instanceof Character) {
        System.out.println("Actor A received Character: " + msg);
    }
}
```

```

public static void main(String[] args) {
    ActorSystem system = ActorSystem.create("MyActorSystem");
    ActorRef actorARef = system.actorOf(ActorA.props(), "actorA");

    // Sending various primitive wrapper messages
    actorARef.tell(123, ActorRef.noSender()); // Integer
    actorARef.tell(true, ActorRef.noSender()); // Boolean
    actorARef.tell((byte) 1, ActorRef.noSender()); // Byte
    actorARef.tell((short) 25, ActorRef.noSender()); // Short
    actorARef.tell(9876543210L, ActorRef.noSender()); // Long
    actorARef.tell(3.14f, ActorRef.noSender()); // Float
    actorARef.tell(2.718, ActorRef.noSender()); // Double
    actorARef.tell('A', ActorRef.noSender()); // Character
}

```

```

ActorA [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (Mar 24, 2025, 1:23:54 PM)
[2025-03-24 13:23:55,165] [INFO] [akka.event.slf4j.Slf4jLogger] [MyActorSystem-1] Actor A received Integer: 123
Actor A received Boolean: true
Actor A received Byte: 1
Actor A received Short: 25
Actor A received Long: 9876543210
Actor A received Float: 3.14
Actor A received Double: 2.718
Actor A received Character: A
[2025-03-24 13:23:55,310] [INFO] [akka.actor.RepointableActorRef] [MyActorSystem-1] Actor A received Integer: 123

```

Explanation: This Akka actor program defines ActorA, which can receive and process various primitive wrapper types such as Integer, Boolean, Byte, Short, Long, Float, Double, and Character. The createReceive method specifies handlers for each data type, directing them to the onMessage method. Inside onMessage, the actor identifies the message type, prints it, and performs additional actions such as terminating the system if it receives an Integer value of 0 or forwarding a message to ActorB if it receives another Integer. The main method initializes an actor system and sends test messages of different types to ActorA.

3. Create 2 Akka Actor classes "ActorA" and "ActorB" to demonstrate the Akka API `setReceiveTimeout()`. ActorA will generate a random integer number from 1 to 5 in a loop for 100 times, and send this integer as a message to ActorB, which would then call `Thread.sleep()` that many seconds. Set the receive timeout to 2 seconds, and when the timeout triggers, send ActorB a `stop()` message, and then create a new instance of ActorB to process the next number.

Main.java

```
package com.example;

import akka.actor.ActorRef;
import akka.actor.ActorSystem;

public class Main {
    public static void main(String[] args) {
        ActorSystem system = ActorSystem.create("ActorSystem");

        ActorRef actorARef = system.actorOf(ActorA.props(), "actorA");

        actorARef.tell("start", ActorRef.noSender());

        try {
            System.out.println("Press ENTER to terminate the system.");
            System.in.read();
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            system.terminate();
        }
    }
}
```

MessageA.java

```
package com.example;

public class MessageA {

    public final int number;

    public MessageA(int number) {
        this.number = number;
    }
}
```

ActorA.java

```
public class ActorA extends AbstractActor {

    private ActorRef actorBRef;
    private final Random random = new Random();

    public static Props props() {
        return Props.create(ActorA.class, ActorA::new);
    }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
            .match(ReceiveTimeout.class, this::onReceiveTimeout)
            .match(Terminated.class, this::onTerminated)
            .match(String.class, this::onStart)
            .build();
    }

    @Override
    public void preStart() {
        actorBRef = getContext().actorOf(ActorB.props(), "actorB");
        getContext().watch(actorBRef);
        getContext().setReceiveTimeout(Duration.create(2, "seconds"));
    }

    private void onStart(String message) {
        for (int i = 0; i < 100; i++) {
            int number = random.nextInt(5) + 1;
            System.out.println("ActorA: Sending number " + number + " to ActorB");
            actorBRef.tell(new MessageA(number), getSelf());

            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }

    private void onReceiveTimeout(ReceiveTimeout timeout) {
        System.out.println("Timeout occurred. ActorB will stop.");
        actorBRef.tell("stop", getSelf());
    }

    private void onTerminated(Terminated terminated) {
        System.out.println("ActorB has been terminated, creating new ActorB instance.");
        actorBRef = getContext().actorOf(ActorB.props(), "actorB");
        getContext().watch(actorBRef);
    }
}
```

ActorB.java

```
package com.example;

import akka.actor.AbstractActor;
import akka.actor.ActorRef;
import akka.actor.Props;

public class ActorB extends AbstractActor {

    public static Props props() {
        return Props.create(ActorB.class, ActorB::new);
    }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
            .match(MessageA.class, this::onMessageA)
            .match(String.class, this::onStop) // Handle stop message
            .build();
    }

    private void onMessageA(MessageA msg) {
        System.out.println("ActorB: Received number " + msg.number + ", sleeping for " + msg.number + " seconds.");
        try {
            Thread.sleep(msg.number * 1000); // Sleep for the number of seconds received
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        // Send "stop" message back to ActorA after processing the message
        getSender().tell("stop", getSelf());
    }

    private void onStop(String message) {
        if (message.equals("stop")) {
            System.out.println("ActorB: Stopping.");
            getContext().stop(getSelf()); // Stop itself
        }
    }
}
```

Output:

```
[2025-03-24 13:45:27,558] [INFO] [akka.event.slf4j.Slf4jLogger] [ActorSystem-akka.actor
Press ENTER to terminate the system.
ActorA: Sending number 3 to ActorB
ActorB: Received number 3, sleeping for 3 seconds.
ActorA: Sending number 3 to ActorB
ActorA: Sending number 3 to ActorB
ActorA: Sending number 1 to ActorB
ActorA: Sending number 1 to ActorB
ActorA: Sending number 3 to ActorB
ActorB: Received number 3, sleeping for 3 seconds.
ActorA: Sending number 2 to ActorB
ActorA: Sending number 4 to ActorB
ActorA: Sending number 4 to ActorB
ActorA: Sending number 3 to ActorB
ActorA: Sending number 4 to ActorB
ActorA: Sending number 5 to ActorB
ActorB: Received number 3, sleeping for 3 seconds.
ActorA: Sending number 3 to ActorB
```

Explanation: In this implementation, ActorA generates random numbers and sends them to ActorB. ActorB processes the numbers by sleeping for the corresponding number of seconds. If ActorA doesn't receive a response within 2 seconds (due to the timeout), it sends a "stop" message to ActorB and creates a new instance of ActorB. The process continues for 100 iterations, and each time ActorB is stopped and replaced with a new instance after the timeout occurs. The output reflects this behavior, showing ActorA sending messages, ActorB sleeping for the specified durations, and new ActorB instances being created after each timeout.